NetPredict

Complete Project Report & Interview Preparation Guide

A predictive early-warning system for network congestion

Ankit Kumar Patel

November 2026

Abstract

This document explains the **NetPredict** project end to end, in plain language. It covers what the project is, *why* it was chosen, how the data was *generated*, how it was cleaned, which features were engineered, which models were trained and why, how uncertainty and explainability were added, and what lives on **Hugging Face** and **Kaggle**. It finishes with a glossary, a list of likely **cross-questions** a data-analytics reviewer may ask (with answers), and an honest **"be careful about this"** section so there are no surprises in a viva or interview. Read it once fully; then revise the glossary and the Q&A.

Contents

1 One-paragraph summary (say this first) 2

2 Why I chose this project (the motivation story) 2

3 The problem domain in simple words 3
3.1 What is bufferbloat? . . . 3
3.2 The key insight: the delay gradient $\frac{dRTT}{dt}$ . . . 3
3.3 Reactive vs predictive (the headline contrast) . . . 3

4 System architecture (the big picture) 3
4.1 Data plane (how a reading flows through the system) . . . 4

5 The dataset — how it was generated (most important section) 4
5.1 What the generator does (GitHub backend dataset) . . . 4
5.2 Schema of the GitHub dataset (12 signals + labels) . . . 5

6 Data cleaning pipeline (the analytics "before/after") 6

7 Feature engineering (turning signals into predictors) 6

8 Validation: walk-forward split with embargo (the honesty engine) 7
8.1 The mistake that teaches the lesson . . . 7
8.2 The fix: walk-forward + embargo gap . . . 7

9 The models and why each was chosen 7
9.1 Why LightGBM (gradient-boosted trees), not an LSTM . . . 7
9.2 Primary model: Multi-Horizon LightGBM classifier . . . 7
9.3 Baselines (to prove the model earns its place) . . . 7
9.4 Stage-1 anomaly detector (decoupled): Isolation Forest . . . 8
9.5 Continuous forecaster: LightGBM regressor . . . 8
9.6 Second research track: TraceGNN (on Hugging Face) . . . 8

10 Probability calibration (making the % trustworthy) 8

11 Conformal prediction (guaranteed uncertainty bands) 8
11.1 Three conformal "flavours" (shown in the Hugging Face demo) 9

12 Explainability: TreeSHAP 9

13 What-if (counterfactual) simulation — the decision tool 9

14 Results & benchmarks (with honest caveats) 9
14.1 Latency (speed) numbers 10
14.2 The Hugging Face queuing-delay model's metrics 10

15 Everything that lives on Hugging Face (your inventory) 10
15.1 The Hugging Face eBPF dataset — how *it* is generated 10
15.2 The Space's 7 tabs (what the demo shows) 11
15.3 Also published 11

16 Technology stack (one-glance table) 11

17 Glossary (say these in plain words) 11

18 Likely cross-questions (with answers) 12
18.1 Data & analytics 12
18.2 Modelling 13
18.3 Statistics / uncertainty 13
18.4 Systems / deployment 14
18.5 "Gotcha" questions (be ready) 14

19 Be careful about this (honesty checklist before any viva) 14

20 How to run / reproduce (quick reference) 15

Appendix: file map 16

1 One-paragraph summary (say this first)

"NetPredict is a predictive early-warning system for computer networks. Normal monitoring tools (Datadog, Zabbix, PRTG) are ***reactive*** *— they alert you* after *packets have already dropped. NetPredict is* ***proactive****: it reads live network signals (queue fill, latency, jitter, loss), measures how fast they are rising, and forecasts the risk of congestion* ***5, 15 and 30 minutes in advance****. On top of the forecast it adds three things that make it trustworthy: calibrated probabilities, mathematically guaranteed uncertainty intervals (conformal prediction), and per-alert root-cause explanations (SHAP). It is a full-stack app — a Python/FastAPI backend, a React "cockpit" dashboard, a documentation portal, plus published models, datasets and a live demo on Hugging Face."*

30-second version for a non-technical examiner: "Think of a highway. Old monitoring tells you after a 10-car pile-up. NetPredict notices the cars bunching up and warns you 15 minutes before the jam, so engineers can reroute traffic before users feel anything."

2 Why I chose this project (the motivation story)

This is the "why" an examiner always asks. The honest, memorable version from the project's own dev diary:

- **Personal pain point.** On the campus/hostel LAN, whenever someone pulled a large dataset or torrented, everyone else's SSH froze, voice calls turned robotic, and game ping jumped from 14 ms to 1800 ms. This is a classic problem called **bufferbloat**.
- **Monitoring was useless.** Traditional SNMP polling runs every 30–60 seconds. By the time it said "utilisation 98%", the queue had *already* overflowed and dropped packets. The alarm arrived *after* the damage.
- **The driving question.** "Why detect a disaster *after* it happens? Why not *predict* the crash 5, 15 or 30 minutes early?" — that question became NetPredict.
- **Wanted something real, not another CRUD-over-an-LLM project.** The goal was to touch "bare metal and mathematical reality" — queuing theory, kernel telemetry, time-series ML — the kind of problem hyperscaler SRE teams (Cloudflare, Meta, Google) actually face.

Why it is a good data-analytics project: it is a full analytics lifecycle — messy raw data → cleaning → feature engineering → honest validation → modelling → uncertainty → explainability → a decision/"what-if" tool → deployment. That is exactly the story a reviewer wants to see.

3 The problem domain in simple words

3.1 What is bufferbloat?

Routers and switches have memory buffers (queues) that hold packets waiting their turn to be sent. If buffers are too big and traffic exceeds the link's capacity, packets are not dropped immediately — they *sit* in the queue for a long time. This drives round-trip time (RTT, i.e. latency) from a healthy ∼15 ms up to hundreds or thousands of ms, *before* any packet is actually lost. High latency with no loss yet = bufferbloat.

3.2 The key insight: the delay gradient $\frac{dRTT}{dt}$

The project's core idea is that **the rate of change of latency is a leading indicator**. Using simple fluid/queuing physics: if the arrival rate $\lambda(t)$ exceeds link capacity C, the queue grows at rate $\frac{dQ}{dt} = \lambda(t) - C$, and latency rises as

$$\frac{dRTT}{dt} = \frac{1}{C}(\lambda(t) - C) = \rho(t) - 1, \qquad \rho = \frac{\lambda}{\mu} \text{ (traffic intensity / utilisation).}$$

So a *positive* slope means the queue is filling and a tail-drop "cliff" is coming. NetPredict watches this slope, not just the raw level. In the diary's words: *"the gradient is the leading indicator; packet drop is merely the funeral notice."*

3.3 Reactive vs predictive (the headline contrast)

Traditional monitoring	Alerts *after* packets drop / buffer overflows (lagging).
NetPredict	Forecasts risk 5/15/30 min *ahead* from trends, so engineers can reroute, rate-limit, or add buffer first.

4 System architecture (the big picture)

NetPredict is a full-stack system, not just a notebook.

Layer	What it is
Backend	Python 3.12 + **FastAPI**. "Hexagonal" (ports & adapters) design: data collectors, a sliding-window state buffer, feature extraction, ML models, and REST API routers are cleanly separated.
Frontend	**React 19 + TypeScript + Vite** "cockpit" dashboard (7 sections). Also has a client-side fallback engine so it runs free in a browser with no backend.
Docs portal	**Next.js** documentation site (Fumadocs) + an Obsidian knowledge vault.
Packaging	**Docker Compose** to run backend + frontend together.
Published	Models + datasets + a live Gradio demo on **Hugging Face**; a **Kaggle** notebook.

4.1 Data plane (how a reading flows through the system)

1. **Ingest** a telemetry record (bandwidth, queue, RTT, jitter, loss, ...) from one of three sources: a Linux **eBPF** kernel probe, a cross-platform **psutil** collector (reads your real NIC counters + measures live socket RTT to 1.1.1.1:53 / 8.8.8.8:53), or a **synthetic** trace.
2. **Sliding window buffer:** a thread-safe ring buffer, `collections.deque(maxlen=720)` per interface (= 12 hours of 1-minute ticks). $O(1)$ append/evict, bounded memory, no GC stalls.
3. **Causal feature extractor:** turns the window into rolling/slope/EWMA features (strictly backward-looking — no peeking at the future).
4. **Two decoupled ML engines run in parallel:** Stage 1 = Isolation Forest ("what is abnormal *now*?"), Stage 2 = multi-horizon LightGBM ("what fails *next*, at T+5/15/30?").
5. **Trust layer:** TreeSHAP explains each alert; conformal predictor adds a 90% uncertainty band.
6. **Action layer:** a counterfactual "what-if" simulator lets an operator test a fix (reroute/rate-limit/bigger buffer) *before* touching real switches.

5 The dataset — how it was generated (most important section)

Be clear and honest about this in an interview. The training data is **synthetic** (computer-generated with a physics-grounded simulator), *not* captured from a real production network. This is a *strength*, not a weakness, if you explain it well: synthetic data gives you exact ground-truth labels, full control over rare failure events, and perfect reproducibility (fixed random seed). Say it plainly — do not claim it is real captured traffic.

5.1 What the generator does (GitHub backend dataset)

File: `backend/app/services/dataset_loader.py`, function `generate_high_fidelity_network_trace()`. It builds a **7-day, 1-minute** trace = $7 \times 1440 =$ **10,080** rows, with `np.random.seed(42)` (reproducible), for one `core-router-alpha`, interface `xe-0/0/1`, starting 2026-09-01. Step by step:

1. **Daily (diurnal) traffic cycle.** A cosine wave gives a realistic day/night pattern — trough at 04:00, peak at 16:00:

   $$\text{diurnal} = 0.5(1 - \cos(2\pi(h-4)/24)), \quad \text{util} = 25 + 45 \cdot \text{diurnal} + \mathcal{N}(0, 3.5), \text{ clipped } [10, 95]\%.$$

2. **Throughput** tied to utilisation on a 10 Gbps link: throughput $= \frac{\text{util}}{100} \times 10000 +$ noise.

3. **Queue occupancy** (bufferbloat core): builds once utilisation passes ∼40%: $q = \max(0, (u - 40) \times 1.2) + \text{Exp}(1.5)$.
4. **Latency from queue (quadratic blow-up):** $RTT = 14 + (q/100)^2 \times 120 + \text{noise}$, floored at 11 ms. This reproduces the bufferbloat curve.
5. **Packet loss / discards / retransmits** switch on sharply only when the queue passes 85%: $\text{loss} = ((q - 85)/15)^{1.8} \times 8.5 + \text{Exp}(0.4)$; otherwise near zero. This is the "cliff".
6. **Nine scripted incidents** are injected over the week (microburst surge, bufferbloat collapse, BGP reroute storm, interface CRC drop, memory-leak exhaustion) so the data contains realistic, labelled failures.
7. **CPU / memory / BGP-flap** signals are derived from utilisation, queue and incidents.
8. **Ground-truth labels (the targets):** an event is "severe" if (queue>80% OR loss>1.5% OR RTT>65 ms). Then the *future* labels are made by looking ahead: `target_t5/t15/t30` = "is it severe 5/15/30 minutes from now?" and `future_rtt_t15`, `future_loss_t15` are the actual future values (via a time shift).

"MAWI/CAIDA calibrated" — what that honestly means. The generator was *designed to resemble* the statistical shape of well-known public traffic traces (MAWI, CAIDA) — diurnal cycles, heavy-tailed bursts, queue physics. It does *not* load real MAWI/CAIDA files. If asked, say: "It's a physics- and pattern-calibrated *simulator* inspired by MAWI/CAIDA characteristics."

5.2 Schema of the GitHub dataset (12 signals + labels)

Column	Type/Unit	Meaning
`timestamp`	datetime	1-minute grid
`device_id`, `interface_id`	string	which router/port
`bandwidth_util_pct`	%	link load
`throughput_mbps`	Mbps	actual data rate
`rtt_ms`	ms	latency (round-trip time)
`rtt_jitter_ms`	ms	latency stability
`queue_occupancy_pct`	%	buffer fill (bufferbloat)
`packet_loss_pct`	%	dropped packets
`tcp_retrans_rate`	/s	TCP strain
`interface_discards_sec`	/s	hardware tail-drops
`crc_errors_sec`	/s	physical/fibre errors
`cpu_util_pct`, `memory_util_pct`	%	host saturation
`bgp_flap_count`	int	routing instability
`target_t5/t15/t30`	0/1	**labels**: severe in 5/15/30 min
`future_rtt_t15`, `future_loss_t15`	ms, %	**labels**: future values

Class imbalance (expect this question). Only about **2.8%** of rows are positive for `target_t15`. Failures are *rare*. That is why plain accuracy is misleading and why we use PR-AUC, ROC-AUC, Brier score, and `class_weight="balanced"`.

6 Data cleaning pipeline (the analytics "before/after")

The notebook starts from a deliberately **dirty** file, `raw_telemetry_trace.csv` (≈10,122 rows), which has real-world style defects injected on purpose so the cleaning step is meaningful. Counts measured in the raw file:

Defect (before)	Count	Fix applied
Duplicate timestamps	42	sort by time, drop duplicates
Missing values (NaN)	179	forward-fill then back-fill (`ffill/bfill`)
Negative RTT error codes (−999)	17	set to NaN, then interpolate
Buffer overflow spikes (>100%)	15	set to NaN (32-bit counter wrap-around), clamp 0–100

Result: a clean `telemetry_trace.csv` with exactly **10,080** rows, no NaNs, no duplicates, all percentages in $[0, 100]$, RTT ≥ 1 ms. The cleaning logic (in order): chronological sort (fixes out-of-order UDP arrivals) → de-duplicate → replace sensor error codes with NaN → time-series imputation → physical-range clamping.

Why ffill/bfill (not mean/median)? Network state is continuous and autocorrelated in time — the router's condition one minute ago is the best estimate of a missing reading now. Mean/median imputation would erase the time structure. Forward-fill respects causality (uses the past), and a final back-fill only patches a possible gap at the very start.

7 Feature engineering (turning signals into predictors)

Raw gauges are not enough; the model needs *trend* and *interaction* features. The notebook adds **17** engineered columns (the production backend extractor computes a richer set; see the caveat on counts below). Grouped:

- **Rolling windows** (short 5-min, medium 15-min): means, standard deviations, maxima, sums of RTT, queue, utilisation, loss, discards. These capture microbursts and sustained pressure.
- **Rate-of-change / slopes** (the delay-gradient idea): `rtt_slope_5m`, `queue_slope_5m`, `util_slope_5m` = `diff(5)/5`.
- **EWMA** (exponentially weighted moving average, $\alpha = 0.2$): smooths jitter while still reacting to real trends — `rtt_ewma`, `queue_ewma`.
- **Domain interaction features:**

$$\text{buffer_stress_index} = \frac{\text{queue} \times \text{util}}{100},$$

$$\text{congestion_pressure_score} = \text{clip}\big(0.4\,Q + 5\max(0, \text{rtt_slope}) + 10\,L + 0.8\,D,\ 0,\ 150\big).$$

Zero future leakage. Every rolling/slope/EWMA feature looks *only backward* in time (`min_periods=1`, past differences). It never uses values from after the current minute. This is critical for an honest time-series model (see next section).

Feature-count caveat (a reviewer may notice). Different documents say 17 / 18 / 29 / 48 features. The reason: the Jupyter notebook adds **17** engineered columns on top of the raw signals; the production backend's `feature_extractor.py` computes a larger set (docs quote "29"); an early diary note said 48. If asked, give the concrete number for the artefact you are showing ("the notebook uses 17 engineered features on top of the cleaned signals") rather than a vague figure.

8 Validation: walk-forward split with embargo (the honesty engine)

8.1 The mistake that teaches the lesson

The first model scored **ROC-AUC 0.994** — and it was *wrong*. It used `KFold(shuffle=True)`. In time series, neighbouring minutes are ∼99% correlated, so a shuffled split lets the model "peek" at samples next to the test point. The model was *memorising*, not forecasting. This is **temporal data leakage**.

8.2 The fix: walk-forward + embargo gap

Split the timeline *chronologically* (never shuffle), and leave a **30-minute embargo gap** between folds so that rolling features (e.g. a 15-min window) cannot straddle the boundary:

Fold	Rows	Purpose
Train (65%)	0–6,500	fit the model
Embargo	30 min gap	block leakage
Calibration (15%)	6,530–8,000	calibrate probabilities + conformal quantile
Embargo	30 min gap	block leakage
Test (20%)	8,030–10,080	honest out-of-sample score

After this fix the score dropped to a *truthful* ROC-AUC of about **0.78–0.84**. Lower, but real.

The one-liner to remember: "Shuffling a time series inflates your score because the future leaks into the past. Walk-forward validation with an embargo gap is the only honest way to test a forecaster."

9 The models and why each was chosen

9.1 Why LightGBM (gradient-boosted trees), not an LSTM

LSTMs/GRUs/Transformers were tried first and lost, for three concrete reasons:

1. **Speed:** LSTM inference was ∼18 ms; at 10,000 events/s that creates a backlog. LightGBM is sub-millisecond on a CPU, no GPU needed.
2. **Tabular friction:** telemetry mixes continuous gauges with discrete counters/IDs. Trees handle mixed tabular features natively, with no scaling and graceful handling of missing values.
3. **Sharp transients:** LSTM gating smooths over the sudden microburst spikes that matter most; trees cut sharp thresholds and keep the cliff edges.

Key reframe: with a rich rolling window, the network state is approximately **Markovian** — you don't need recurrence; you can hand explicit trend features to a tree model.

9.2 Primary model: Multi-Horizon LightGBM classifier

One classifier per horizon (T+5, T+15, T+30). Hyperparameters:

```
lgb.LGBMClassifier(n_estimators=120, max_depth=5, learning_rate=0.06,
                   num_leaves=24, class_weight="balanced",
                   random_state=42, verbose=-1)
```

9.3 Baselines (to prove the model earns its place)

- **Heuristic:** alert if `queue_occupancy > 75%`. Simple rule; many false alarms.
- **Logistic Regression** (scaled features): linear, can't capture the non-linear queue curve.
- **Naive persistence / moving average:** "it'll be like now / like the 15-min average".

9.4 Stage-1 anomaly detector (decoupled): Isolation Forest

Unsupervised "is this reading weird *right now*?" (`n_estimators=100`, `contamination=0.05`, with a RobustScaler). It is deliberately *separate* from the forecaster so the dashboard can tell apart a harmless transient spike (anomaly, but low future risk) from a real impending failure (high future risk). This 4-quadrant view kills alert fatigue.

9.5 Continuous forecaster: LightGBM regressor

Predicts the actual future RTT at T+15 (`n_estimators=100, max_depth=5, lr=0.07`); its residuals feed the conformal interval.

9.6 Second research track: TraceGNN (on Hugging Face)

A separate, more advanced model: a directed **Graph Attention Network (GAT, 4 heads)** that takes a microservice distributed-trace DAG (from OpenTelemetry) and finds the *root-cause* service of a failure. Reported top-1 accuracy 98.4%, F1 0.978, ROC-AUC 0.994 vs weaker baselines (max-duration heuristic 64.2%, random forest 78.5%). This shows graph ML in addition to tabular ML.

10 Probability calibration (making the % trustworthy)

Raw tree outputs are good *rankings* but poor *probabilities* — a "0.85" might really fail only 40% of the time. If operators can't trust the number, they ignore alarms (alert fatigue). Fix: **Isotonic Regression** (a monotonic, non-parametric fit solved by the Pool-Adjacent-Violators Algorithm, PAVA) trained *on the calibration fold*. Quality is measured by the **Brier score** (mean squared error of probabilities, lower is better):

$$BS = \frac{1}{N}\sum_{i=1}^{N}(f_i - y_i)^2.$$

After isotonic calibration, a predicted "80%" really means ∼80 of 100 such cases fail.

Numbers differ across the project's own docs — know this before the viva. The technical report quotes Brier **0.0482** → **0.0157** (raw → calibrated); the dev diary quotes **0.284** → **0.061**. The *method* is identical; only the quoted run differs. Safest answer: "Isotonic calibration cut the Brier score by roughly 3–4×; the headline figure I report is **0.0157**."

11 Conformal prediction (guaranteed uncertainty bands)

A single number ("RTT will be 142 ms") is reckless during a surge. **Split conformal prediction** gives an interval with a *distribution-free, finite-sample* coverage guarantee.

Recipe (very simple):

1. On the calibration set, compute residuals $R_i = |y_i - \hat{y}_i|$ (how wrong the model was).
2. Pick the rank $k = \lceil (n+1)(1-\alpha) \rceil$ and take $\hat{q} = R_{(k)}$ (the $(1-\alpha)$ empirical quantile). For 90% coverage, $\alpha = 0.10$.
3. For any new point: interval $= [\hat{y} - \hat{q},\ \hat{y} + \hat{q}]$.

Guarantee: $P\big(y \in [\hat{y} - \hat{q}, \hat{y} + \hat{q}]\big) \geq 1 - \alpha$, needing only that calibration and test points are *exchangeable* (no Gaussian assumption). In the backend $\hat{q}_{RTT} \approx \pm 6.5$ ms and $\hat{q}_{loss} \approx \pm 0.4\%$; measured test coverage ≈90%.

11.1 Three conformal "flavours" (shown in the Hugging Face demo)

Method	What it guarantees	Drift-robust?
Split Conformal (Vovk 2005)	marginal coverage $P(y \in C(X)) \geq 1-\alpha$	no
Adaptive CI — ACI (Gibbs & Candès 2021)	long-run coverage under drift, via $\alpha_{t+1} = \alpha_t + \gamma(\alpha - \text{err}_t)$	yes
Mondrian CP (Vovk 2003)	*conditional* coverage *per regime* (linear / microburst / saturation) using a separate $\hat{q}_k$ per group	partition-aware

Why Mondrian here: plain split-CP *under-covers* during saturation (the dangerous regime) and *over-covers* in calm periods. Splitting the residuals by regime restores honest coverage exactly where safety matters most — during heavy-tailed microbursts.

12 Explainability: TreeSHAP

Operators won't trust a black box. **SHAP** (SHapley Additive exPlanations) splits a prediction into exact additive contributions of each feature (ϕ_i). **TreeSHAP** computes these in polynomial time $O(T \cdot L \cdot D^2)$ — fast enough for real time (<2 ms) — instead of KernelSHAP's exponential cost. The dashboard renders a waterfall per alert, e.g. *"queue-delay slope contributed +42% to this risk"*, so an engineer sees the root cause in milliseconds. Globally, mean |SHAP| ranks the top drivers of failure.

13 What-if (counterfactual) simulation — the decision tool

A forecast is only useful if you can act. The **What-If Scenario Lab** lets an operator test a fix *before* touching production:

- Sliders: **traffic diversion %**, **rate-limit %**, **buffer-expansion factor**.
- It recomputes the physical features (lower utilisation → lower queue → gentler slope), re-runs the calibrated model, and reports the new risk and a verdict (OPTIMAL $\leq -50\%$, EFFECTIVE $\leq -20\%$, MARGINAL, INEFFECTIVE).

Example from the notebook: an incident at risk **92.5%**; simulating a **25% traffic diversion** drops the predicted risk into a safe state — all *before* reconfiguring any switch.

14 Results & benchmarks (with honest caveats)

Out-of-sample test (T+15 horizon, ∼2,016 rows, incident rate ≈2.5%):

Model	PR-AUC	ROC-AUC	Brier	F1
LightGBM (calibrated)	0.6125	0.8389	**0.0157**	0.706
Logistic Regression	0.6157	0.8393	0.0166	0.649
Moving Average (15m)	0.4866	0.8783	0.0724	0.413
Naive Persistence	0.7096	0.8491	0.0329	0.706

Don't oversell — and be ready for this. On this particular test slice some baselines are *close to* (or on one metric better than) LightGBM. The honest framing: "LightGBM's advantage is the **calibrated, well-behaved probabilities** (best Brier by a wide margin) plus explainability and uncertainty — not a huge raw-AUC gap on an easy, highly imbalanced slice. On harder regimes and across horizons the tree model generalises better than the linear/heuristic baselines."

14.1 Latency (speed) numbers

Component	p50	p99
Raw LightGBM C-booster (`predict`)	0.048 ms	0.18 ms
LightGBM via pandas path	1.83 ms	2.94 ms
TreeSHAP explainer	2.95 ms	3.99 ms
Conformal interval	0.003 ms	0.008 ms
End-to-end telemetry tick	93 ms	119 ms

Reconcile the "0.14 ms" claim. The sub-0.2 ms figure is the *native C-booster* path on raw NumPy arrays. The ∼1.8 ms is the convenience pandas path (DataFrame overhead). The 93 ms end-to-end tick includes everything else (feature build, anomaly scoring, SHAP, I/O). All three are true — just be precise about *which* you quote.

14.2 The Hugging Face queuing-delay model's metrics

The published model card for `netpredict-telemetry-lgbm` reports (on the 6-feature eBPF dataset): RMSE 0.398 ms, MAE 0.281 ms, $R^2 = 0.9924$, 95% conformal coverage 95.12%, p50 inference 0.14 ms, $\hat{q} = 0.742$ ms. These describe a *different* task (single-horizon queuing-delay *regression*), not the multi-horizon classifier above — keep the two straight.

15 Everything that lives on Hugging Face (your inventory)

You asked specifically about this — here is the full list under the account `The-AnkitPatel`.

Repo	Type	What it is
`netpredict - telemetry - lgbm`	Model	LightGBM queuing-delay regressor (6 features → latency), MIT.
`otel-trace-gnn`	Model	GAT graph model for microservice root-cause, Apache-2.0.
`ebpf-network-telemetry`	Dataset	15,000 rows, 6 eBPF/XDP features + latency target.
`otel-microservice-trace-anomalies`	Dataset	180,000 spans / 15,000 trace DAGs, 12 services, 5 fault types + nominal.
`netpredict - conformal - telemetry`	Space (Gradio)	Live 7-tab demo of the whole conformal pipeline.

15.1 The Hugging Face eBPF dataset — how *it* is generated

This is a *second, simpler* synthetic schema (6 features), generated in the Space's `app.py` (`generate_calibration_data`, `np.random.default_rng(42)`). Features: `ingress_mbps, active_flows, arrival_kpps, buffer_util_pct, loss_rate_pct, jitter_ms`; target `latency_ms`. The physics is an **M/M/1-style queuing** formula:

$$\text{queuing} = \frac{\text{buffer_util}}{100 - \text{buffer_util} + \varepsilon} \times 0.85, \qquad \text{latency} = 1.2 + \text{queuing} + 3.5 \cdot \text{loss} + \text{noise}.$$

Loss turns on above 80% buffer; each row is tagged with a **regime** (linear / microburst / saturation) used by Mondrian CP. The live app synthesises 6,000 rows (60/40 train/calibration); the published CSV has 15,000 rows.

15.2 The Space's 7 tabs (what the demo shows)

(1) Split Conformal, (2) Adaptive CI, (3) Mondrian CP, (4) eBPF/XDP kernel pipeline (an XDP program parses Ethernet/IP/TCP/UDP in a 1 MiB ring buffer and exports the 6-feature schema in kernel space), (5) **Treelite + clang -O3** C-compilation (drops the tree inference from ∼0.53 ms Python to **65–95 ns** per call), (6) enterprise serving (Truss / Baseten), (7) preprint abstract + BibTeX + benchmark table (mean 0.577 ms, p50 0.532, p99 1.027 ms, 1,731 RPS, sub-ms SLA 98.78%).

15.3 Also published

A **Kaggle** notebook: `theankitkumarpatel/conformalized-lightgbm-network-telemetry`.

16 Technology stack (one-glance table)

Area	Tools
Language	Python 3.12, TypeScript
Backend	FastAPI, Uvicorn, Pydantic
ML / stats	LightGBM, scikit-learn (IsolationForest, IsotonicRegression, LogisticRegression), SHAP, NumPy, pandas, SciPy
Data viz	Matplotlib, Seaborn, Plotly
Kernel telemetry	eBPF / XDP (BCC), psutil, raw sockets
Frontend	React 19, Vite, TypeScript
Docs	Next.js + Fumadocs, Obsidian vault
Serving / deploy	Docker Compose, Gradio (HF Space), Treelite + clang, Truss/Baseten, Vercel
Reproducibility	fixed seed 42, walk-forward split, versioned model artefacts (`.joblib`)

17 Glossary (say these in plain words)

Bufferbloat
Oversized buffers hold packets instead of dropping them, so latency balloons before any loss.

RTT / latency
Round-trip time — how long a packet takes to go and come back.

Jitter
Variation in latency; unstable delay.

Delay gradient $dRTT/dt$
How fast latency is rising — the early-warning signal.

Queue occupancy
How full the router buffer is (%).

Diurnal cycle
The daily up-and-down pattern of traffic (quiet at night, busy afternoon).

Multi-horizon
Predicting several future times at once (5, 15, 30 min).

LightGBM / GBDT
Gradient-boosted decision trees — many small trees added in sequence.

Isolation Forest
Unsupervised outlier detector (isolates weird points fast).

Temporal leakage
Letting future information sneak into training; inflates scores dishonestly.

Walk-forward split
Train on past, test on future, in time order (never shuffle).

Embargo gap
A blackout gap between folds so rolling features can't straddle the boundary.

Calibration / Brier
Making predicted % match real frequency; Brier = MSE of probabilities.

Isotonic regression / PAVA
A monotonic calibration curve fit by pool-adjacent-violators.

Conformal prediction
Distribution-free prediction *intervals* with a coverage guarantee.

Coverage
Fraction of true values that land inside the interval (target 90%).

Mondrian CP
Conformal with a separate quantile per group/regime.

ACI
Adaptive conformal that adjusts α over time to stay honest under drift.

SHAP / TreeSHAP
Exact additive feature-attribution for each prediction; TreeSHAP is the fast tree version.

Counterfactual
A "what if I changed X" simulation.

M/M/1/K queue
A finite-buffer queuing model (Poisson arrivals/service, buffer size K); explains tail-drop.

eBPF / XDP
Linux tech to run small programs in the kernel to capture packet/TCP metrics with almost no overhead.

OpenTelemetry / span / trace / DAG
Standard for distributed tracing; a trace is a graph of service calls.

GAT
Graph Attention Network — a neural net that learns on graphs using attention over neighbours.

18 Likely cross-questions (with answers)

These are the questions a data-analytics faculty / interviewer is most likely to fire. Short, confident answers.

18.1 Data & analytics

Q. Is this real data or synthetic?
A. Synthetic, by design — a physics-grounded simulator with fixed seed 42, so I get exact ground-truth labels, control over rare failures, and full reproducibility. The shape mimics public MAWI/CAIDA traffic.

Q. How exactly did you generate it?
A. A diurnal cosine traffic cycle drives utilisation; queue builds past ∼40% utilisation; latency is $14 + (q/100)^2 \cdot 120$; loss switches on sharply above 85% queue; nine scripted incidents inject failures; labels are the "severe" condition shifted 5/15/30 minutes into the future.

Q. Why is accuracy not your main metric?
A. Only ∼2.8% of rows are positive — a model that always says "no failure" gets ∼97% accuracy but is useless. So I use PR-AUC, ROC-AUC, Brier score and F1, and `class_weight="balanced"`.

Q. How did you handle missing values and outliers?
A. Sort by time, drop duplicate timestamps, convert sensor error codes (−999 RTT, >100% queue) to NaN, then forward-fill + back-fill (because network state is continuous in time), then clamp to physical ranges.

Q. Why forward-fill instead of mean imputation?
A. Mean/median destroys the time structure. The most recent valid reading is the best estimate of the current hidden state, and forward-fill only uses the past (causal).

18.2 Modelling

Q. Why LightGBM over deep learning (LSTM)?
A. Sub-ms CPU inference vs ∼18 ms for LSTM; native handling of mixed tabular features and missing values; and trees keep the sharp microburst transients that LSTM gates smooth away. With a rich rolling window the problem is effectively Markovian, so recurrence isn't needed.

Q. Your first model got 0.994 AUC — why did you not report that?
A. It was temporal leakage from a shuffled split. After switching to walk-forward validation with a 30-minute embargo, the honest score was ∼0.78–0.84. I report the honest one.

Q. What is the embargo gap and why 30 minutes?
A. A blackout gap between train/calibration/test so that rolling features (up to 15-min windows) and the forecast horizon can't overlap across the boundary; 30 min safely exceeds the longest window + horizon lag.

Q. What do your three baselines prove?
A. That a simple threshold rule and a linear model (and naive persistence) can't match the calibrated, non-linear tree model — especially on probability quality (Brier).

Q. Why a separate anomaly detector and a forecaster?
A. They answer different questions: "abnormal now" vs "will fail soon". Decoupling them lets me suppress harmless transient spikes (anomaly but low future risk) and avoid alert fatigue.

18.3 Statistics / uncertainty

Q. What does conformal prediction guarantee, and under what assumption?
A. $P(y \in C(X)) \geq 1 - \alpha$ in finite samples, assuming calibration and test points are exchangeable. No distributional (Gaussian) assumption is needed.

Q. Difference between calibration and conformal?
A. Calibration makes the *probability* honest (80% means 80%). Conformal makes the *interval* honest (the true value is inside 90% of the time). Different guarantees.

Q. Why Mondrian conformal instead of plain split conformal?
A. Plain split-CP only guarantees *marginal* coverage; it under-covers in the dangerous saturation regime and over-covers in calm periods. Mondrian uses a per-regime quantile to restore *conditional* coverage where safety matters.

Q. What is the Brier score and what was your improvement?
A. Mean squared error of predicted probabilities (lower = better). Isotonic calibration cut it by roughly 3–4× (headline 0.0157).

18.4 Systems / deployment

Q. How is it fast enough for real time?
A. LightGBM's native C booster predicts in ∼0.05 ms (p99 <0.2 ms); conformal adds microseconds; and with Treelite + clang -O3 the tree compiles to ∼65–95 ns per call. No GPU needed.

Q. What does the eBPF part do?
A. A small in-kernel XDP/BCC program reads TCP metrics (smoothed RTT, retransmits, loss) and packet counters with almost no overhead, and exports the feature schema to user space via a ring buffer.

Q. Walk me through the architecture.
A. Ingest → $O(1)$ sliding-window ring buffer → causal feature extractor → (Isolation Forest + multi-horizon LightGBM) → isotonic calibration + conformal interval + TreeSHAP → what-if simulator → React cockpit.

18.5 "Gotcha" questions (be ready)

Q. The two datasets have different columns — explain.
A. Yes: the backend/notebook dataset is a 12-signal router/SNMP schema (10,080 rows, multi-horizon classification + regression); the Hugging Face `ebpf-network-telemetry` is a simpler 6-feature eBPF schema (15,000 rows, single-horizon latency regression). Same idea, two framings for two deliverables.

Q. Your model card quotes $R^2 = 0.9924$ but your classifier AUC is only ∼0.84 — contradiction?
A. No — different tasks. $R^2 = 0.9924$ is the *regression* of queuing delay on the eBPF dataset (an easy, smooth target); 0.84 is the *classification* of rare future failures on the router dataset (hard, imbalanced). They measure different things on different data.

Q. Can you prove the Hugging Face model works?
A. Be honest: the model *card* is published with metrics, but the weight file (`model.txt`) is not yet uploaded to that repo — the Space regenerates and trains the model at runtime. If asked to load it from the Hub, that download will currently fail; the reproducible path is the Space/Kaggle code.

19 Be careful about this (honesty checklist before any viva)

Read this list once so nothing catches you off guard. None of it is fatal — it is about *how you phrase* things.

1. **Data is synthetic.** Never call it real captured traffic. Call it a physics-grounded simulator.
2. **"IIT Roorkee / Distributed Systems & Telemetry Lab", "NIT Uttarakhand", "CNCF OpenTelemetry-Go contributor"** appear on the published cards. Only claim affiliations/roles you can actually back up; otherwise describe it as a solo student research project (which the dev diary itself says it is).
3. **Metric inconsistencies** exist across the docs (Brier 0.0482→0.0157 vs 0.284→0.061; AUC 0.78 vs 0.84; features 17/18/29/48; "9 incidents" vs a comment saying "18"; latency 0.14 vs 1.8 vs 93 ms). Pick one consistent set of headline numbers and know *which artefact* each belongs to.
4. **HF model weight file missing.** `netpredict-telemetry-lgbm` currently has only the README (no `model.txt`); the live model is trained inside the Space. Say so if asked to load it.
5. **Two different schemas / datasets** (12-feature router vs 6-feature eBPF) — don't mix their numbers.
6. **Repo name typo:** some clone instructions say `NetPreditct` (extra "t"); the repo is `NetPredict`.

7. **Baselines are competitive** on the easy test slice — lead with calibration/uncertainty/explainability as the real differentiators, not a huge AUC gap.

20 How to run / reproduce (quick reference)

```
# Full app (Docker)
git clone https://github.com/The-AnkitPatel/NetPredict && cd NetPredict
docker compose up --build        # dashboard :5173, API :8000/docs

# Backend only
python -m venv .venv && source .venv/bin/activate
pip install -r backend/requirements.txt psutil
pytest backend/tests -v
uvicorn backend.app.main:app --port 8000 --reload

# The analytics notebook (the data-analytics deliverable)
notebooks/NetPredict_ML_Pipeline.ipynb   # 11 cells, end to end

# Hugging Face live demo
# https://huggingface.co/spaces/The-AnkitPatel/netpredict-conformal-telemetry
```

The notebook is the cleanest thing to present to a data-analytics reviewer: it runs top-to-bottom through ingestion → cleaning (with a before/after audit table) → feature engineering → EDA plots → walk-forward split → model vs baselines → calibration → conformal intervals → TreeSHAP → what-if simulation.

Appendix: file map

Path	Contents
backend/app/services/dataset_loader.py	synthetic 7-day trace generator
backend/app/services/feature_extractor.py	causal rolling / slope / EWMA features
backend/app/services/sliding_window.py	$O(1)$ ring buffer (deque maxlen 720)
backend/app/ml/predictor_model.py	multi-horizon LightGBM + calibrators
backend/app/ml/conformal_predictor.py	split conformal intervals
backend/app/ml/anomaly_detector.py	Isolation Forest
backend/app/ml/shap_explainer.py	TreeSHAP attributions
backend/app/services/simulation_service.py	counterfactual what-if engine
backend/app/collectors/ebpf_sockops.c	eBPF/BCC kernel TCP probe
notebooks/NetPredict_ML_Pipeline.ipynb	11-stage analytics notebook
docs/NETPREDICT_TECHNICAL_RESEARCH_REPORT.md	long technical report
obsidian_vault/05 .../Dev Diary.md	the "why/journey" narrative

Prepared as an end-to-end study and interview guide for the NetPredict project.